

Universidad Veracruzana

Desarrollo de Sistemas Web (DSW-19559)

Evolución de la web (1.0 a 4.0) y stacks de la experiencia educativa

Comparativo, línea de tiempo y candidatos del banco de proyectos

Producto individual (R01)

Alejandro Pacheco Luna

Equipo DSW-E01

Facilitador: Dr. Gabriel Rodríguez Vásquez

Orizaba, Ver., 25 de septiembre de 2026

1. Para qué sirve este comparativo

Antes de elegir un proyecto y de decidir con qué tecnología se construye cada incremento, conviene tener claro qué cambió entre una etapa de la web y la siguiente: quién hace qué (el servidor, el navegador, el usuario, los dispositivos), por dónde viajan los datos y qué problemas de seguridad aparecen. Este documento resume esa evolución, la relaciona con los cuatro stacks que usamos en la experiencia educativa y compara dos fichas del banco C11 como candidatas. La comparación no reserva ningún proyecto; la selección se confirma en el foro F02.

Escribo desde la experiencia concreta de nuestro equipo: ya construimos el incremento Web 1.0 (JSP/Servlets) y el Web 2.0 (JSF/PrimeFaces) del PR09, así que varios ejemplos vienen de ahí.

2. Matriz comparativa de las etapas

Dimensión	Web 1.0	Web 2.0	Web 3.0	Web 4.0
Propósito	Publicar y consultar información; formularios básicos	Participar: el usuario crea y modifica contenido; aplicaciones interactivas	Separar cliente y servicios; datos con significado y reutilizables por máquinas	Adaptarse al contexto: sensores, eventos y decisiones asistidas
Arquitectura	Servidor genera HTML completo por petición (CGI, JSP/Servlets)	Servidor con componentes y estado de vista; actualizaciones parciales por AJAX	Cliente SPA en el navegador + API REST/JSON sin estado + base de datos	Fuentes de eventos (dispositivos o simuladores) + broker/colas + servicios que interpretan
Interacción	Recarga completa; un formulario, una respuesta	Componentes ricos, validación en servidor con respuesta parcial	Interfaz reactiva; el cliente decide qué pedir y cuándo	La aplicación reacciona sin que el usuario pida: alertas, telemetría
Papel del usuario	Lector; llena formularios	Autor y colaborador; sesión autenticada	Usuario de una aplicación que consume varios servicios	Participante en un entorno instrumentado; supervisa y corrige
Flujo de datos	Navegador -> servidor -> HTML	Navegador <-> servidor (postback AJAX) -> base	Navegador <-> API (JSON) -> base; varios clientes por API	Dispositivo -> broker/API -> base -> vistas y alertas
Seguridad típica	Validar entrada, evitar SQL inyectado, sesión de contenedor	ViewState contra CSRF, control de sesión, escape de salida	Tokens (JWT/OAuth), CORS, validación en la API, HTTPS	Autenticar dispositivos, etiquetar procedencia, no confundir simulación con medición real
Tecnologías	HTML, HTTP, CGI, PHP, JSP/Servlets	AJAX, JSF/PrimeFaces, jQuery, XML/JSON	Angular/React, REST, Spring Boot, JSON, OAuth	MQTT, WebSockets, series de tiempo, reglas/ML
En nuestra EE	P02: JSP + Tomcat 9 + PostgreSQL	P03: JSF 2.3 + PrimeFaces 12 + Tomcat 9	Angular 16 + Spring Boot 2.7 + PostgreSQL	Simulador IoT (MQTT) + PostgreSQL

Las fechas y los nombres de etapa son una convención (Aghaei, Nematbakhsh y Farsani, 2012; O'Reilly, 2005), no un estándar: una aplicación real mezcla rasgos de varias etapas. Nuestro PR09, por ejemplo, ya etiqueta la procedencia de cada lectura desde Web 1.0 pensando en la etapa 4.0.

3. Línea de tiempo



Figura 1. Evolución de la web y su correspondencia con los incrementos de la EE.

Qué cambia de una etapa a otra, en una frase cada una: de 1.0 a 2.0 el usuario pasa de leer a escribir y el servidor deja de reconstruir toda la página en cada clic; de 2.0 a 3.0 la interfaz se muda al navegador y el servidor se vuelve una API que otros clientes también pueden usar; de 3.0 a 4.0 la fuente de datos ya no es solo una persona frente a un formulario, sino dispositivos que publican eventos y una aplicación que los interpreta.

4. Etapa, arquitectura y responsabilidades

Web 1.0. El servidor hace todo: recibe la petición, consulta la base y devuelve HTML. En nuestro P02, el Servlet coordinaba y la JSP presentaba; la validación vivía en clases Java y respondía 400 con la lista de errores. El usuario solo ve páginas completas; cada envío es una recarga. Es simple de razonar y de probar con curl, pero cada interacción cuesta una página entera.

Web 2.0. El servidor sigue mandando, pero mantiene el estado de la vista y responde a peticiones parciales. En P03 el bean de vista guarda lo capturado, PrimeFaces envía el formulario por AJAX y solo se actualizan mensajes, formulario y tabla. Aparecen responsabilidades nuevas: proteger el estado (ViewState), controlar la sesión con una persona autenticada y no con un rol elegido en un combo, y evitar que un componente redondee en silencio lo que el servidor debería rechazar. Ya no basta curl para probar: hay cookies, ViewState y respuestas parciales.

Web 3.0. La interfaz se construye en el navegador (Angular) y el servidor expone una API (Spring Boot) que no guarda sesión: cada petición lleva su token. El usuario interactúa con una aplicación que se siente local; el servidor responde JSON con códigos 2xx y 4xx que el cliente debe mostrar. La misma API podría servir a una app móvil.

Web 4.0. La aplicación recibe eventos de dispositivos (o de un simulador, en nuestro caso) con instante, variable, unidad y valor, los guarda y los interpreta con las mismas reglas de umbral que ya existen. El usuario supervisa y anota; la responsabilidad ética es no presentar una simulación como si fuera una medición real.

5. Comparación de los stacks de la EE

Aspecto	JSP / Servlets + Tomcat	JSF 2.3 / PrimeFaces + Tomcat	Angular + Spring Boot	IoT simulado (MQTT) + PostgreSQL
Dónde se dibuja la interfaz	Servidor (JSP)	Servidor (Facelets), con actualización parcial	Navegador (componentes Angular)	Vistas de las etapas anteriores más paneles de eventos
Estado	Sesión HTTP y parámetros	Beans @ViewScoped/@SessionScoped, ViewState	Estado en el cliente; API sin estado	Series de lecturas en la base; estado del dispositivo
Validación	Clases Java, respuesta 400	Validadores y conversores JSF, mensajes por campo	Cliente (formularios reactivos) y servidor (Bean Validation), 4xx	Rango físico y procedencia al ingerir el evento
Seguridad	Sesión del contenedor, PreparedStatement	ViewState contra CSRF, filtro de sesión, escape en Facelets	JWT/OAuth, CORS, HTTPS	Autenticación del publicador, etiquetado simulado/real
Persistencia	JDBC parametrizado	JDBC parametrizado (reutilizado)	JPA/JDBC en Spring Data	Inserción por eventos; índices por sensor y fecha
Cuándo conviene	Aprender el ciclo HTTP; sistemas pequeños	Formularios ricos con equipo Java; intranets	Interfaces complejas y varios clientes	Monitoreo, telemetría, alertas
Coste que vimos	Recarga completa; sin componentes	Curva de CDI/scopes; jars extra (Weld, JAXB) en Tomcat 9	Dos proyectos y dos despliegues; versiones de Node/Angular	Necesita simulador y reglas claras de procedencia

Con PostgreSQL como base común, lo que se conserva entre etapas es el modelo y las reglas: en nuestro caso, las siete tablas, la transacción de lectura más alerta y los validadores pasaron de P02 a P03 sin cambios. Lo que cambia es la capa de presentación y la forma de asegurar la sesión.

6. Dos candidatos del banco C11

Comparo las dos fichas que más se parecen en patrón (lectura, umbral, alerta) y que el equipo consideró viables. La comparación no reserva el proyecto.

Criterio	PR09 Monitoreo de huerto o ambiente	PR03 Monitoreo de laboratorio de cómputo
Problema	Las observaciones ambientales no se conservan con contexto ni permiten reconocer tendencias y alertas	Incidencias y condiciones ambientales sin relación con equipos, horarios y mantenimiento
Usuarios (3 roles)	Responsable de huerto, observador, coordinación	Encargado de laboratorio, soporte técnico, coordinación
Flujo principal	Elegir sensor -> capturar lectura -> validar -> guardar con instante y procedencia -> alerta si sale del umbral -> historial	Registrar equipo/incidencia -> lectura ambiental -> historial y alerta
Requisitos	Zonas, variables, lecturas, umbrales, historial, alertas y anotaciones (RF01-RF07); API y IoT después	Inventario, incidencias, estado, lecturas, historial y alertas; API y IoT después
Entidades	zona, variable, lectura, umbral, alerta, anotación (+ sensor)	equipo, laboratorio, incidencia, lectura, alerta, usuario

Viabilidad técnica	Alta: variables físicas simples, reglas claras, simulación sin hardware	Media: mezcla inventario e incidencias con telemetría; complejidad media-alta
Riesgo	Presentar simulación como medición real -> etiquetar procedencia	Usar telemetría para vigilar personas -> limitar a condiciones del espacio

Recomendación razonada: PR09. Cubre el mismo patrón que PR03 con un dominio más acotado, su riesgo se controla con una decisión de diseño (la columna de procedencia) y cada etapa de la web le agrega una capacidad observable sin inventar otro sistema: captura (1.0), formularios y sesión (2.0), API y panel (3.0), sensores simulados (4.0). PR03 queda como segunda opción por compartir el patrón, aunque arrastra el inventario de equipos.

7. Conclusiones propias

Lo que más me quedó claro al pasar de P02 a P03 es que el cambio de etapa no es cambiar de librería: es mover responsabilidades. En Web 1.0 el servidor era responsable de todo y un curl bastaba para probar; en Web 2.0 el servidor guarda estado por vista y hay que probar con un navegador porque el formulario ya no es un POST plano. Lo que no se movió fue la base y las reglas, y eso es lo que hace posible que el proyecto sea acumulativo.

También aprendí que la seguridad cambia de forma con cada etapa: en 1.0 me preocupaba el SQL inyectado, en 2.0 la sesión y el ViewState, y para 3.0 y 4.0 ya sé que vendrán tokens y la pregunta de si un dato viene de un sensor real o de un simulador. Elegir el stack no es elegir el más moderno, sino el que reparte las responsabilidades como el problema lo necesita.

Bibliografía

- Aghaei, S., Nematbakhsh, M. A. y Farsani, H. K. (2012). Evolution of the World Wide Web: From Web 1.0 to Web 4.0. *International Journal of Web & Semantic Technology*, 3(1), 1-10. <https://doi.org/10.5121/ijwest.2012.3101>
- Berners-Lee, T., Cailliau, R., Luotonen, A., Nielsen, H. F. y Secret, A. (1994). The World-Wide Web. *Communications of the ACM*, 37(8), 76-82. <https://doi.org/10.1145/179606.179671>
- Berners-Lee, T., Hendler, J. y Lassila, O. (2001). The Semantic Web. *Scientific American*, 284(5), 34-43.
- O'Reilly, T. (2005). *What Is Web 2.0: Design Patterns and Business Models for the Next Generation of Software*. O'Reilly Media. <https://www.oreilly.com/pub/a/web2/archive/what-is-web-20.html>
- Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures* (tesis doctoral). University of California, Irvine. <https://ics.uci.edu/~fielding/pubs/dissertation/top.htm>
- IETF (2022). RFC 9110: HTTP Semantics. <https://www.rfc-editor.org/rfc/rfc9110>
- Apache Software Foundation. *Apache Tomcat 9 Documentation*. <https://tomcat.apache.org/tomcat-9.0-doc/>
- Eclipse Foundation. *Jakarta Servlet 4.0 Specification*. <https://jakarta.ee/specifications/servlet/4.0/>
- Eclipse Foundation. *Jakarta Server Faces 2.3 Specification*. <https://jakarta.ee/specifications/faces/2.3/>
- PrimeTek. *PrimeFaces 12 Documentation*. https://primefaces.github.io/primefaces/12_0_0/
- Angular. *Version compatibility*. <https://angular.dev/reference/versions>
- VMware. *Spring Boot 2.7.18 Reference Documentation*. <https://docs.spring.io/spring-boot/docs/2.7.18/reference/html/>
- OASIS (2019). *MQTT Version 5.0*. <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>

PostgreSQL Global Development Group. PostgreSQL 16 Documentation. <https://www.postgresql.org/docs/16/>

OWASP Foundation. Application Security Verification Standard 5.0. <https://owasp.org/www-project-application-security-verification-standard/>

W3C (2023). Web Content Accessibility Guidelines (WCAG) 2.2. <https://www.w3.org/TR/WCAG22/>

Rodríguez Vásquez, G. (2026). C11. Banco de proyectos integradores; M02 y M03, Desarrollo de Sistemas Web. Universidad Veracruzana (material del curso en Eminus).